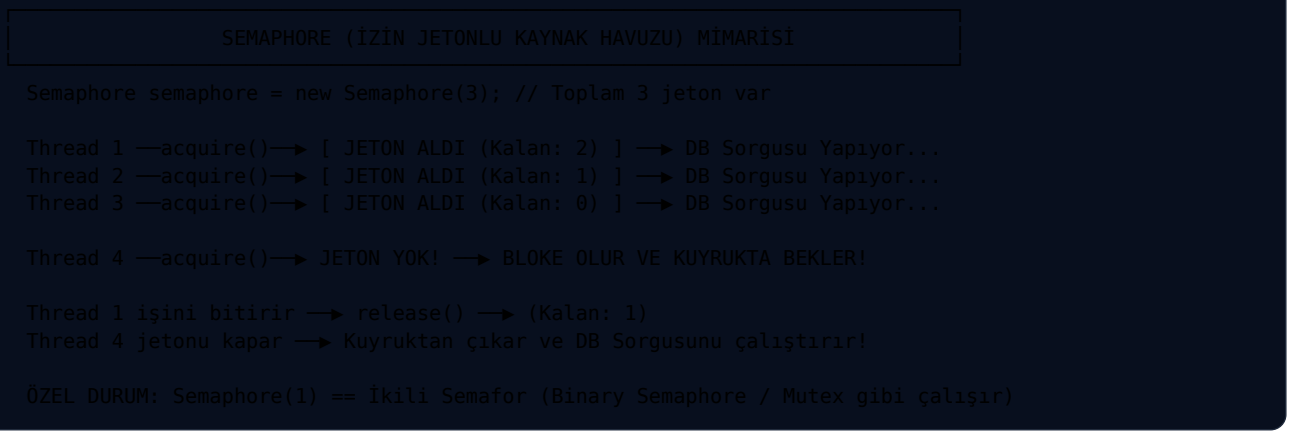
22

BÖLÜM: İLERI DÜZEY SENKRONIZASYON & KILIT-SIZ ARAÇLAR | MODULE: ADVANCED SYNCHRONIZATION & LOCK-FREE PRIMITIVES

Semaphore: İzin Tabanlı Kaynak Havuzları & Backpressure Sınırlama

Semaphore: Permitted Resource Pooling & Backpressure Throttling

⚡ Bellek & Eşzamanlılık Akış Şeması ⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir gece kulübünün otoparkını düşünün. Otoparkta sadece 3 araçlık yer var (Semaphore permits = 3). İlk gelen 3 araba içeri girer ve güvenlik bariyeri kapanır. 4. araba kapıda bekler. İçerideki arabalardan biri çıkış yaptığı anda (`release()`), kapıdaki arabaya yeşil ışık yanar (`acquire()`) ve içeri girer. Semafor, pahalı bir kaynağı (veritabanı bağlantısı, harici API kotası) aynı anda aşırı yüklenmeden koruyan jetonlu güvenlik turnikesidir!

🇬🇧 ENGLISH (ELI5)

Think of a nightclub coat check with exactly 3 plastic hangers (`Semaphore(3)`). The first 3 patrons receive hangers and enter. The fourth patron waits in line at the counter. The moment an existing patron checks out and returns their hanger (`release()`), the clerk hands it to the waiting patron (`acquire()`). A Semaphore acts as a token-based rate-limiter, throttling concurrent access to bounded resources (database pools, third-party REST quotas) to prevent system brownouts.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Semafor Mimarisi ve Mutex Farkı: 1) AQS State = İzin Sayısı: Semaphore, AQS'in `state` alanını kalan izin (permit) sayısı olarak kullanır. 2) Semaphore vs Mutex (Önemli Mülakat Farkı!): Bir Mutex kilitte kilidi KİM ALDIYSA YALNIZCA O BIRAKABİLİR (Ownership prensibi). Ancak Semaphore'da SAHİPLİK YOKTUR! Thread A `acquire()` çağırıp izni alabilir; Thread B ise `release()` çağırarak o izni havuza geri verebilir! Bu özellik thread'ler arası sinyalleşmede kullanılır. 3) Backpressure ve Rate Limiting: Veritabanına aynı anda en fazla 20 sorgu gitmesini istiyorsanız, arkaya 1000 thread gelse bile `semaphore.tryAcquire(timeout)` ile veritabanı OOM ve çökmeden korunur.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Semaphore Internals & Mutex Distinction: 1) AQS Permit Representation: AQS `state` holds the available permit count. `acquire()` executes a CAS loop decrementing state; if state <= 0, the thread parks on the AQS shared sync queue. 2) Crucial Ownership Difference vs Mutex: A Mutex or `ReentrantLock` has strict ownership: only the thread that acquired the lock may release it. A Semaphore HAS NO CONCEPT OF OWNERSHIP. Thread 1 can invoke `acquire()`, and Thread 2 can invoke `release()`! 3) Dynamic Permit Inflation: Calling `release()` without preceding acquires actually INCREASES permits beyond initial capacity, a feature used in reactive backpressure signaling but a hazardous bug if done accidentally.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Semaphore ile Veritabanı Bağlantı Havuzu Throttling
import java.util.concurrent.Semaphore;
import java.util.concurrent.TimeUnit;

public class DatabaseConnectionThrottler {
    // Veritabanı maksimum 3 eşzamanlı bağlantıya izin veriyor
    private final Semaphore semaphore = new Semaphore(3, true); // Adil (Fair)

    public void executeQuery(String queryName) {
        try {
            // 1 saniye içinde izin alamazsa vazgeç (Fail-Fast)
            if (semaphore.tryAcquire(1, TimeUnit.SECONDS)) {
                try {
                    IO.println("Bağlantı alındı: " + queryName + " yürütülüyor...");
                    Thread.sleep(300); // DB işlemi simülasyonu
                } finally {
                    semaphore.release(); // İzni havuza geri ver
                    IO.println("Bağlantı bırakıldı: " + queryName);
                }
            } else {
                IO.println("⚠ BAĞLANTI HAVUZU DOLU! " + queryName + " reddedildi (Backpressure).");
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: Mülakatçı 'Bir semafora fazladan release() çağrılırsa ne olur?' diye sorar. Yanıt: 'Hata fırlatmaz! İzin sayısı başlangıçtaki kapasitenin üzerine çıkar (Permit Inflation)! Örneğin 3 izinli semafora 5 kere release() derseniz artık 5 thread aynı anda içeri girebilir. Bu durum beklenmedik kaynak tükenmelerine yol açabilir.'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Subtle Interview Trap: 'What happens if you invoke `semaphore.release()` more times than `acquire()`?' Answer: 'No exception is thrown! The permit count increases monotonically past the initial constructor capacity (Permit Inflation). Calling release() 5 times on a Semaphore(3) raises available permits to 5, potentially overwhelming downstream resources.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Semaphore, eşzamanlı erişilebilecek kaynak sayısını (permits) sınırlar.
- Semaforlarda sahiplik (ownership) yoktur; bir thread alıp başka thread bırakabilir.
- İkili Semafor (Binary Semaphore / permits=1), mutex alternatifi olarak kullanılabilir.

🎯 Key Takeaways for the Interview

- Semaphore limits the maximum number of concurrent threads accessing a resource.
- Semaphores have no thread ownership; Thread A can acquire and Thread B can release.
- A binary semaphore (permits=1) functions as a non-reentrant mutual exclusion lock.